

Project 3: SQL Data Analysis

DecodeLabs Industrial Training Kit · Batch 2026 · Prepared by Bhumi Singh · Database: orders.db (SQLite) · 1,200 records

1. Project Overview

This project applies SQL to the same cleaned e-commerce orders dataset used in Projects 1 and 2 (1,200 records, 14 fields), loaded into a SQLite database (orders.db). 21 queries were written and executed, covering SELECT, WHERE filtering, GROUP BY aggregation, HAVING, ORDER BY, and COUNT/SUM/AVG functions, plus business-focused queries like revenue contribution percentages and repeat-customer identification.

2. Understanding SQL Execution Order

Although SQL is written as SELECT → FROM → WHERE → GROUP BY → ORDER BY, the database engine actually executes it as: **FROM/JOIN** → **WHERE** → **GROUP BY** → **HAVING** → **SELECT** → **ORDER BY**. This explains the “Alias Trap”: a column alias created in SELECT (e.g. TotalPrice AS rev) cannot be referenced inside WHERE, because WHERE runs before SELECT and the alias does not exist yet. It CAN, however, be used in ORDER BY, since ORDER BY runs after SELECT.

```
-- This FAILS (alias used before it exists):
SELECT TotalPrice AS rev FROM orders WHERE rev > 1000;

-- This WORKS (condition repeated in WHERE, alias used only in ORDER BY):
SELECT TotalPrice AS rev
FROM orders
WHERE TotalPrice > 1000
ORDER BY rev DESC;
```

3. Query Highlights & Results

3.1 Total Revenue by Product (SUM + GROUP BY + ORDER BY)

```
SELECT Product, COUNT(*) AS num_orders, SUM(TotalPrice) AS total_revenue
FROM orders
GROUP BY Product
ORDER BY total_revenue DESC;
```

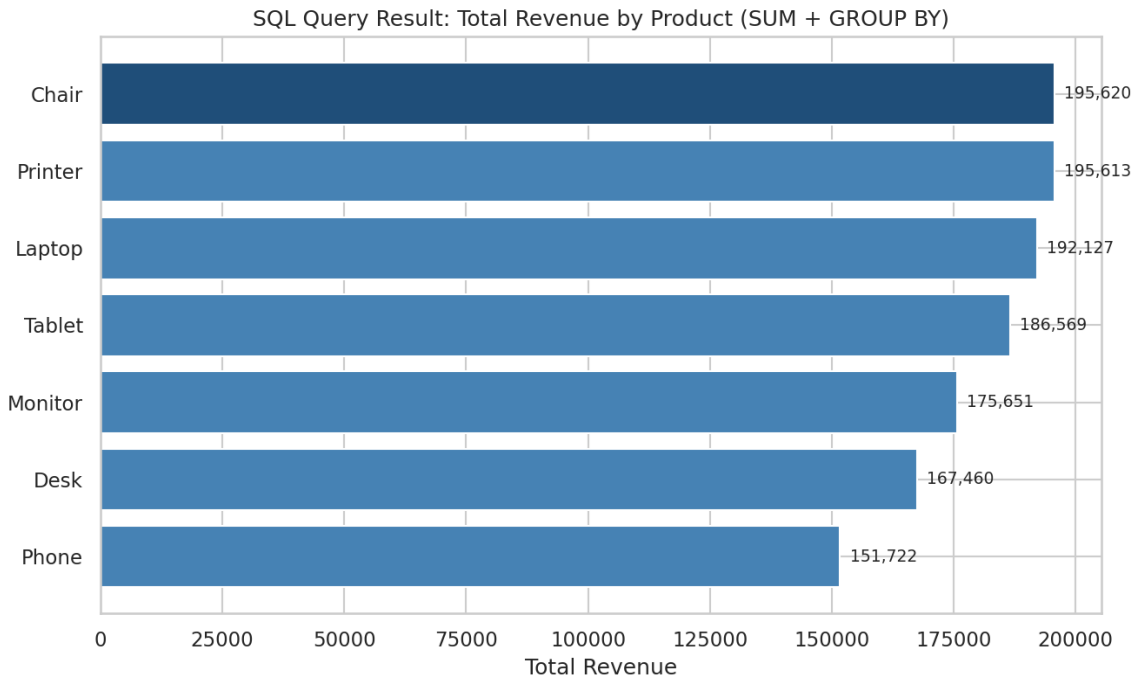


Fig 1: Chair leads revenue at Rs. 195,620, narrowly ahead of Printer.

3.2 Average Order Value by Payment Method (AVG + GROUP BY)

```
SELECT PaymentMethod, COUNT(*) AS num_orders, ROUND(AVG(TotalPrice),2) AS avg_order_value
FROM orders
GROUP BY PaymentMethod
ORDER BY avg_order_value DESC;
```

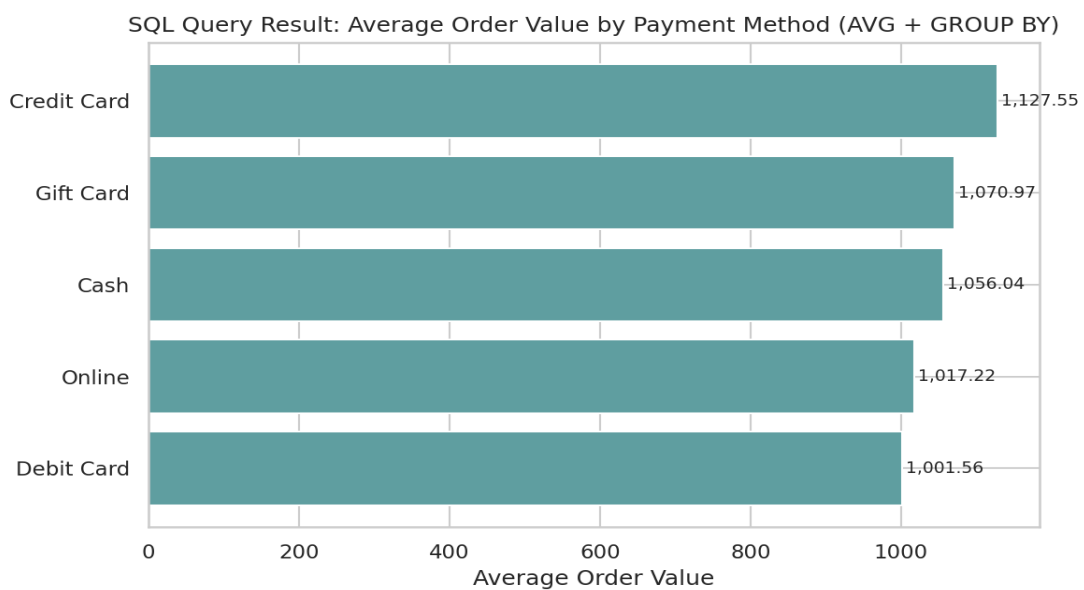


Fig 2: Credit Card orders carry the highest average value at Rs. 1,127.55.

3.3 Monthly Revenue Trend (SUM + GROUP BY on date substring)

```
SELECT substr(Date,1,7) AS year_month, COUNT(*) AS num_orders, SUM(TotalPrice) AS total_revenue
FROM orders
GROUP BY year_month
ORDER BY year_month;
```

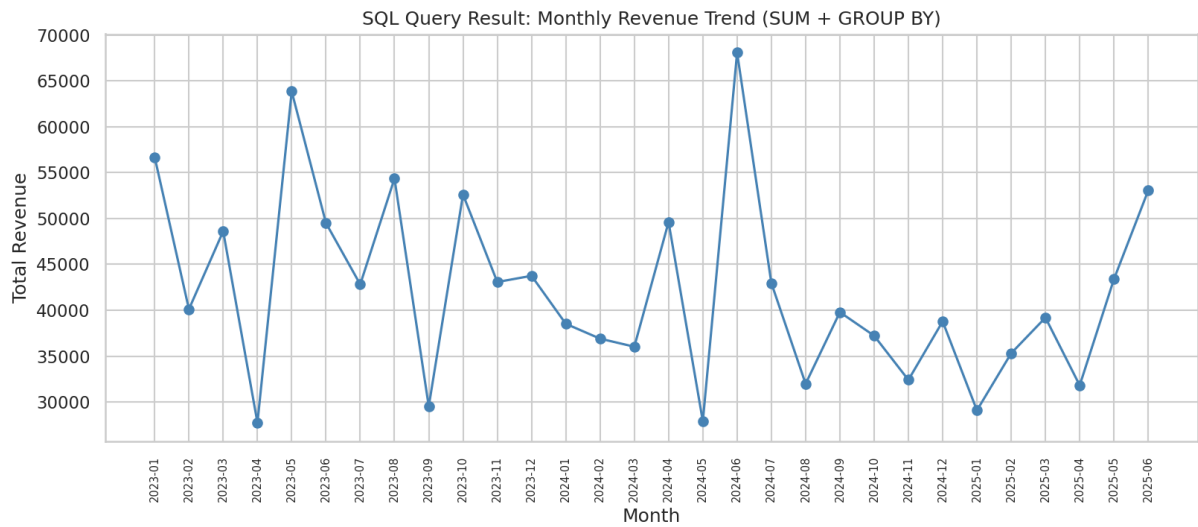


Fig 3: Monthly revenue trend, Jan 2023 to Jun 2025.

3.4 Filtering Aggregated Results with HAVING

```
SELECT Product, SUM(TotalPrice) AS total_revenue
FROM orders
GROUP BY Product
HAVING SUM(TotalPrice) > 180000
ORDER BY total_revenue DESC;
```

Product	Total Revenue
Chair	195,620.11
Printer	195,612.61
Laptop	192,126.56
Tablet	186,568.95

HAVING filters groups AFTER aggregation (unlike WHERE, which filters individual rows before grouping) — only products clearing Rs. 180,000 in total revenue appear here.

4. Key Business Insights

1. **Chair** and **Printer** are statistically tied for top revenue generator (Rs. 195,620 vs Rs. 195,613) — both deserve equal inventory priority.
2. **41.4% of all orders** are Cancelled or Returned, calculated directly via SQL CASE/SUM logic — confirming the same finding from the Project 2 EDA and validating the analysis across two different methods.
3. **Credit Card** orders have the highest average value (Rs. 1,127.55); **Debit Card** the lowest (Rs. 1,001.56) — a 12.6% spread worth investigating for targeted payment-method promotions.
4. Revenue is fairly evenly distributed across products (12–15.5% each) — no single category dominates, which lowers business risk from any one product underperforming.
5. Only **11 customers** placed more than 1 order in the dataset — repeat purchase rate is low, suggesting an opportunity for loyalty/retention campaigns.
6. Coupon usage has a small POSITIVE association with order value (Rs. 1,057.64 used vs Rs. 1,043.37 not used) — consistent with the Project 2 finding that discounts are not eroding margin via smaller baskets.

Full set of 21 queries available in queries.sql. Complete raw output for every query is available in outputs/Project3_SQL_Results.txt.

Prepared as part of the DecodeLabs Industrial Training Kit (Batch 2026) — Project 3 checkpoint submission.